

대규모 언어 모델과 PDF 파싱 기술을 활용한 지능형 문서 분석 자동화



1. 서론: 지식 노동의 자동화와 AI 연구원의 탄생



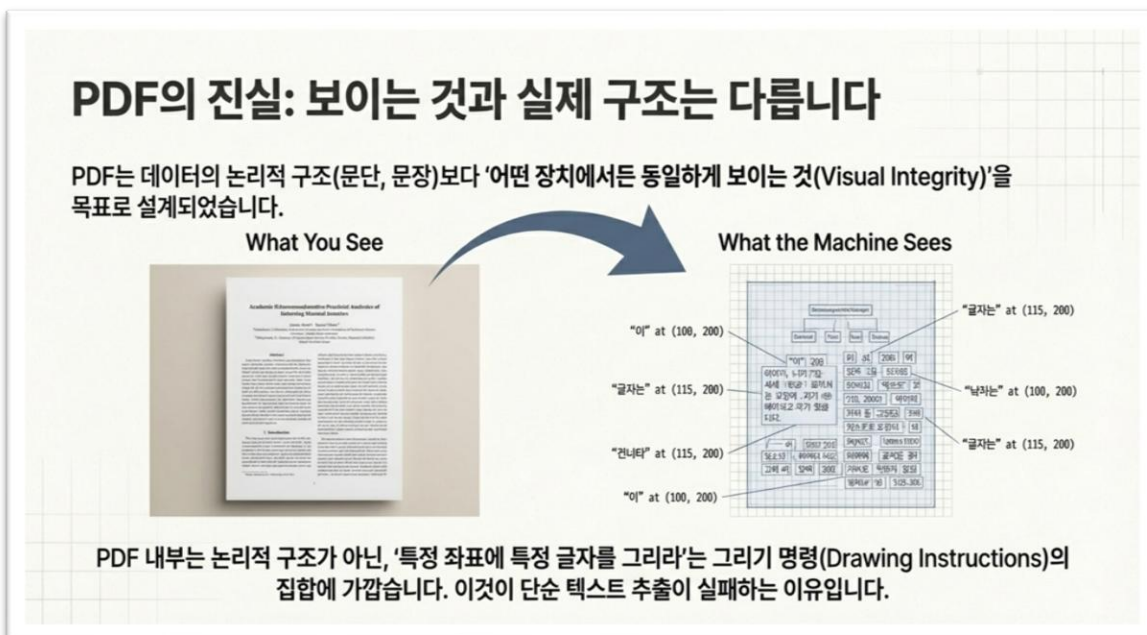
디지털 트랜스포메이션(Digital Transformation)이 가속화되는 현대 사회에서 정보의 생산량은 기하급수적으로 증가하고 있습니다. 매일 수천 편의 학술 논문, 수만 건의 기업 보고서, 그리고 셀 수 없는 정책 문서들이 PDF(Portable Document Format) 형태로 쏟아져 나옵니다. 이러한 정보의 홍수 속에서 핵심 지식을 신속하게 추출하고 이해하는 능력은 연구자, 학생, 그리고 지식 노동자에게 있어 생존을 위한 필수 역량이 되었습니다. 그러나 인간의 인지 능력은 물리적 한계를 지니고 있으며, 방대한 텍스트 데이터를 일일이 독해하고 분석하는 과정은 막대한 시간과 정신적 에너지를 요구합니다.

문제를 해결하기 위해, 최신 인공지능 기술인 대규모 언어 모델(Large Language Model, LLM)과 정교한 문서 처리 기술을 결합하여 '나만의 AI 연구원'을 구축하는 과정을 심도 있게 다룹니다. 단순한 코딩 실습을 넘어 배후에 있는 기술적 원리, 데이터 엔지니어링의 정수, 그리고 프롬프트 엔지니어링의 미묘한 예술까지 포괄적으로 분석합니다.



우리는 왜 상용 서비스를 쓰지 않고 직접 개발해야 할까요? 시중에는 이미 'ChatPDF'와 같은 훌륭한 서비스들이 존재합니다. 그러나 범용 서비스는 데이터 보안(Privacy), 비용 효율성(Cost Efficiency), 그리고 커스터마이징(Customization) 측면에서 명확한 한계를 가집니다. 기업의 기밀 문서를 외부 서버에 업로드할 수 없는 보안 환경, 특수한 레이아웃을 가진 기술 문서를 처리해야 하는 상황, 혹은 수천 개의 문서를 일괄 처리해야 하는 대규모 파이프라인 구축의 필요성은 우리에게 '직접 개발할 수 있는 능력'을 요구합니다. 본 강의는 Python 생태계의 강력한 PDF 처리 라이브러리인 PyMuPDF와 OpenAI의 GPT API를 활용하여, 이러한 요구사항을 완벽하게 충족시키는 엔드투엔드(End-to-End) 시스템을 구축하는 것을 목표로 합니다.

2. PDF 문서의 구조적 이해와 텍스트 추출 기술의 이론



PDF 파일을 단순히 '읽는 것'과 기계가 이해할 수 있는 데이터로 '변환하는 것' 사이에는 거대한 기술적 장벽이 존재합니다. PDF는 본래 데이터의 구조적 저장보다는 '어떤 장치에서든 동일하게 보이는 것(Visual Integrity)'을 목표로 설계된 포맷이기 때문입니다. 따라서 PDF 내부에는 '문단', '문장', '표'와 같은 논리적 구조 정보가 명시적으로 존재하지 않는 경우가 많으며, 대신 "이 글자는 좌표 (100, 200)에 위치한다"와 같은 그리기 명령(Drawing Instructions)의 집합으로 구성되어 있습니다.

2.1 PDF 처리 라이브러리 비교 및 PyMuPDF의 선정 이유

모든 PDF 라이브러리가 같지는 않습니다

라이브러리	특징	장점	단점
PyPDF2 (pypdf)	순수 Python 구현	설치 용이, 의존성 적음, 파일 조작 강점	텍스트 추출 성능 낮음, 레이아웃 분석 기능 부족
PDFMiner	텍스트 분석 중심	폰트, 위치 등 상세 데이터 추출 가능	속도 매우 느림, 사용법 복잡
PyMuPDF (fitz)	C 라이브러리 바인딩	압도적인 처리 속도, 정밀한 좌표 기반 추출	C 의존성으로 인한 설치 환경 제약 (드림)



압도적인 속도 (Overwhelming Speed):
C 라이브러리(MuPDF) 기반으로 타 라이브러리 대비 수십 배 빠른 성능을 보여 대용량 문서 처리에 필수적입니다.



정밀한 좌표 기반 추출 (Precise Coordinate-based Extraction):
복잡한 레이아웃 분석에 핵심적인 텍스트의 정확한 좌표(BBox) 정보를 제공합니다.

Python 환경에서 PDF를 다루기 위한 라이브러리는 다양하지만, 각기 다른 장단점을 지니고 있습니다. 성공적인 시스템 구축을 위해서는 도구의 특성을 정확히 이해하고 선택하는 것이 첫 번째 단계입니다.

PyMuPDF를 핵심 엔진으로 채택합니다. PyMuPDF는 Artifex Software가 유지 관리하는 고성능 렌더링 엔진인 MuPDF의 Python 바인딩으로, 텍스트 추출뿐만 아니라 이미지 렌더링, 주석(Annotation) 처리, 그리고 저수준(Low-level) 데이터 접근까지 지원하는 올인원(All-in-One) 솔루션입니다. 특히 대용량 논문을 처리할 때 타 라이브러리 대비 수십 배 빠른 속도를 보여주며, 복잡한 레이아웃 분석에 필수적인 텍스트의 좌표(BBox) 정보를 정확하게 제공한다는 점이 결정적입니다.

2.2 PyMuPDF 설치 및 환경 설정

PyMuPDF 를 사용하기 위해서는 Python 환경에 패키지를 설치해야 합니다. 과거에는 fitz 라는 이름으로 불렸으나, 현재는 PyMuPDF 라는 패키지명으로 통합 관리되고 있습니다. 그러나 코드 내에서 모듈을 불러올 때는 여전히 fitz 또는 pymupdf 를 사용한다는 점에 유의해야 합니다.

가상 환경 생성 및 활성화 권장

```
python -m venv venv
```

Windows: venv\Scripts\activate, Mac/Linux: source venv/bin/activate

패키지 설치 (강의 기준 버전 1.25.2 이상 권장)

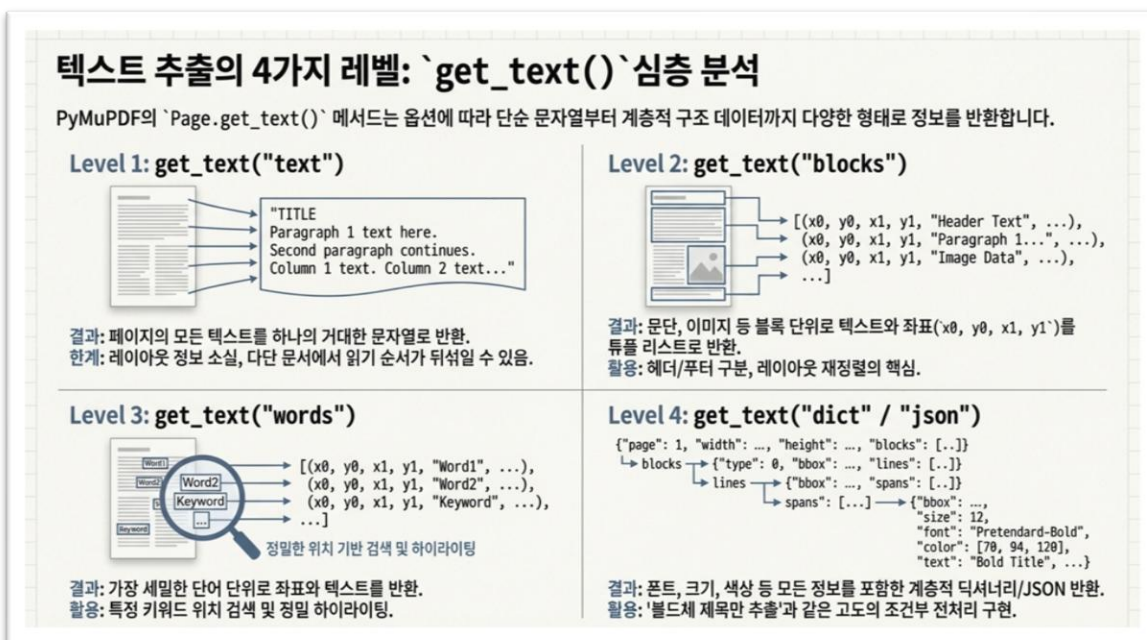
```
pip install PyMuPDF
```

설치가 완료되면 Python 스크립트에서 다음과 같이 임포트하여 버전을 확인할 수 있습니다.

```
import pymupdf
print(pymupdf.__version__)
```

2.3 텍스트 추출의 층위(Layer)와 get_text() 메서드 심층 분석

PyMuPDF의 Page.get_text(option) 메서드는 옵션에 따라 다양한 형태의 데이터를 반환합니다. 단순한 문자열 추출을 넘어, 문서의 구조를 복원하기 위해서는 각 옵션의 반환 데이터 구조를 명확히 이해해야 합니다.



2.3.1 단순 텍스트 추출 ("text")

가장 기본적인 형태로, 페이지의 모든 텍스트를 하나의 문자열로 반환합니다.

```
text = page.get_text("text")
```

- **특징:** 줄바꿈 문자(\n)로 행을 구분합니다.
- **한계:** 레이아웃 정보가 소실되며, 다단(Multi-column) 문서의 경우 왼쪽 단과 오른쪽 단의 텍스트가 섞여서 출력될 수 있습니다. 읽는 순서(Reading Order)가 시각적 순서와 일치하지 않을 수 있습니다.

2.3.2 블록 기반 추출 ("blocks")

문단이나 이미지와 같은 '블록' 단위로 데이터를 추출합니다.

```
blocks = page.get_text("blocks")
```

-
- **반환 구조:** (x0, y0, x1, y1, "text", block_no, block_type) 튜플의 리스트.
 - **활용:** 각 블록의 좌표 (x0, y0, x1, y1)를 알 수 있으므로, 이를 기반으로 본문, 헤더, 사이드바 등을 구분하거나 다단 레이아웃을 재정렬하는 데 핵심적인 역할을 합니다.

2.3.3 단어 기반 추출 ("words")

가장 세밀한 단위인 '단어' 수준의 정보를 추출합니다.

```
words = page.get_text("words")
```

-
- **반환 구조:** (x0, y0, x1, y1, "word", block_no, line_no, word_no)
 - **활용:** 특정 키워드의 위치를 찾거나(Search), 문장 내에서 정밀한 하이라이팅을 할 때 사용됩니다.

2.3.4 딕셔너리/JSON 추출 ("dict", "json")

페이지의 모든 정보를 계층적 구조(Block -> Line -> Span -> Char)를 가진 딕셔너리나 JSON 형태로 반환합니다.

- **활용:** 폰트 종류, 크기, 색상 정보를 포함하므로, "볼드체로 된 제목만 추출"하거나 "폰트 크기 10pt 미만인 주석 제외"와 같은 고도의 전처리 로직을 구현할 때 필수적입니다.

3. 전처리(Preprocessing) 전략: 노이즈 제거와 데이터 정제

"Garbage In, Garbage Out": 노이즈가 AI의 판단을 흐리게 합니다

PDF에서 추출한 날것의 텍스트에는 LLM의 요약 성능을 저해하는 노이즈가 가득합니다. 이는 문맥 파악을 방해하고, 비용을 증가시키며, 환각(Hallucination)의 원인이 됩니다.

- **반복되는 헤더:** "한국농공학회논문집 66권 4호"
- **페이지 번호와 푸터:** 문장 중간에 삽입된 숫자나 투고 정보

Before (Raw Text)

안정적인 생산기반을 마련할
필요 **65** 가 제기되고 있다.

After (Clean Text)

안정적인 생산기반을 마련할
필요가 제기되고 있다.

'Garbage In, Garbage Out(쓰레기가 들어가면 쓰레기가 나온다)'은 데이터 과학의 불변의 진리입니다. PDF에서 추출한 날것(Raw)의 텍스트에는 논문의 본문 내용뿐만 아니라, 헤더(Header), 푸터(Footer), 페이지 번호, 워터마크 등 LLM의 요약 성능을 저해하는 노이즈(Noise)가 다수 포함되어 있습니다. 이러한 노이즈는 모델의 문맥 파악을 방해하고, 불필요한 토큰을 소모하여 비용을 증가시키며, 때로는 환각(Hallucination)의 원인이 되기도 합니다.

3.1 노이즈의 유형과 영향 분석

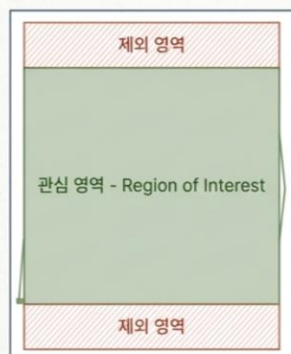
첨부된 예시 논문('과정기반 작물모형...')을 분석해보면 다음과 같은 노이즈 패턴을 식별할 수 있습니다.

1. **반복되는 헤더:** "한국농공학회논문집 66 권 4 호"와 같은 문구가 매 페이지 상단에 반복됩니다. 이를 제거하지 않으면 요약 결과에 "이 논문은 한국농공학회논문집 66 권 4 호라는 내용을 담고 있다"는 무의미한 문장이 포함될 수 있습니다.
2. **페이지 번호와 푸터:** 하단의 "65", "66" 같은 숫자나 "Accepted: May 08, 2024" 같은 투고 정보는 문장의 흐름을 끊습니다. 예를 들어, "안정적인 생산기반을 마련할 필요 **65** 가 제기되고 있다"와 같이 문장 중간에 숫자가 삽입되어 문법적 오류를 유발합니다.
3. **참고문헌 및 메타데이터:** 논문 요약의 목적이 '저자의 주장 파악'이라면, 수십 페이지에 달하는 참고문헌 리스트나 저자의 이메일 주소 등은 요약 대상에서 제외되어야 할 정보입니다.

3.2 영역 기반 클리핑(Cliping) 알고리즘 구현

기하학적 접근법: 불필요한 영역을 잘라내기

가장 효과적인 방법은 문서의 기하학적 영역을 이용하는 것입니다. 헤더와 푸터는 페이지의 상단과 하단에 위치한다는 규칙을 활용하여, 해당 영역을 물리적으로 잘라냅니다.



```
# 텍스트를 추출할 유효 영역(ROI) 설정
# Rect(좌, 상, 우, 하)
clip_rect = pymupdf.Rect(
    0,                                # x0: 왼쪽 끝
    header_height,                    # y0: 상단 여백 제외
    page.rect.width,                  # x1: 오른쪽 끝
    page.rect.height - footer_height # y1: 하단 여백 제외
)

# 설정한 영역 내의 텍스트만 추출
text = page.get_text(clip=clip_rect)
```

가장 직관적이고 효과적인 전처리 방법은 문서의 **기하학적 영역(Geometry)**을 이용하는 것입니다. 헤더와 푸터는 페이지의 상단과 하단 가장자리에 위치한다는 보편적인 규칙을 활용하여, 해당 영역을 물리적으로 잘라내는(Crop) 방식입니다.

PyMuPDF는 `get_text()` 메서드에 `clip` 파라미터를 제공하여 이러한 작업을 효율적으로 수행할 수 있게 합니다. `clip` 파라미터에는 텍스트를 추출할 사각형 영역(Rect 객체)을 전달합니다.⁵

3.2.1 좌표계 계산 로직

PyMuPDF의 좌표계는 페이지의 좌측 상단을 (0, 0)으로 둡니다.

- **페이지 전체 크기:** `page.rect` (너비: `width`, 높이: `height`)
- **헤더 영역:** y 좌표 0 부터 `header_height` 까지.
- **푸터 영역:** y 좌표 `height - footer_height` 부터 `height` 까지.
- **관심 영역(ROI):** y 좌표 `header_height` 부터 `height - footer_height` 까지.

전처리 로직 상세 구현 (01_pdf_without_header_footer.py)

```
import pymupdf
import os
```

```
pdf_file_path = "../data/과정기반 작물모형을 이용한 웹 기반 밀 재배관리 의사결정 지원시스템
```

설계 및 구축.pdf"

```
doc = pymupdf.open(pdf_file_path)
```

```
header_height = 80
```

```
footer_height = 80
```

```
full_text = ""
```

```
for page in doc:
```

```
    rect = page.rect # 페이지 크기 가져오기
```

```
    header = page.get_text(clip=(0, 0, rect.width, header_height))
```

```
    footer = page.get_text(clip=(0, rect.height - footer_height, rect.width, rect.height))
```

```
    text = page.get_text(clip=(0, header_height, rect.width, rect.height - footer_height))
```

```
    full_text += text + '\n-----\n'
```

```
# 파일명만 추출
```

```
pdf_file_name = os.path.basename(pdf_file_path)
```

```
pdf_file_name = os.path.splitext(pdf_file_name)[0] # 확장자 제거
```

```
txt_file_path = f'../output/{pdf_file_name}_with_preprocessing.txt'
```

```
with open(txt_file_path, 'w', encoding='utf-8') as f:
```

```
    f.write(full_text)
```

3.3 고급 전처리: 다단(Multi-column) 레이아웃 대응 전략

다단(Multi-column) 문서의 함정: 뒤섞인 독해 순서

분할 클리핑 (Split Clipping)

페이지가 2단임이 확실하다면, 페이지를 좌측 절반과 우측 절반으로 나누어 각각 텍스트를 추출한 뒤 합치는 것이 가장 확실한 방법입니다.

```
half_width = page.rect.width / 2 # 좌측 영역 클리핑
left_rect = pymupdf.Rect(0, header_height, half_width, page.rect.height - footer_height)
text_left = page.get_text(clip=left_rect) # 우측 영역 클리핑
right_rect = pymupdf.Rect(half_width, header_height, page.rect.width, page.rect.height - footer_height)
text_right = page.get_text(clip=right_rect) # 순서에 맞게 텍스트 결합
full_text = text_left + "\n" + text_right
```

학술 논문의 대다수는 2 단(Two-column) 레이아웃을 채택하고 있습니다. 위의 단순 클리핑 방식은 1 단 문서에는 유효하지만, 2 단 문서에서는 치명적인 문제를 야기할 수 있습니다. `get_text("text")`는 기본적으로 좌측 상단에서 우측 하단으로 텍스트를 읽어나가는데, 이 과정에서 왼쪽 단의 첫 줄을 읽고 바로 오른쪽 단의 첫 줄을 읽어버리는 '순서 섞임(Reading Order Disruption)' 현상이 발생할 수 있습니다.

예를 들어:

- 왼쪽 단: "본 연구는 인공지능을..."
- 오른쪽 단: "활용하여 농업 생산성을..."
- 추출 결과: "본 연구는 인공지능을... 활용하여 농업 생산성을..." (운 좋게 연결됨)
- 하지만 실제로는: "본 연구는 인공지능을 (왼쪽 1 행)... (왼쪽 2 행)... (오른쪽 1 행) 활용하여..."
순서여야 하는데, 줄 단위로 횡단하며 읽으면 문장이 뒤섞입니다.

3.3.1 해결책 1: 블록 정렬 알고리즘 (Block Sorting)

PyMuPDF의 `get_text("blocks")`를 사용하여 텍스트 블록을 추출한 뒤, 파이썬의 정렬 기능을 이용해 논리적 순서를 복원합니다. 일반적으로 2 단 문서의 읽기 순서는 (1) 수평 위치(x 좌표)가 아닌 수직 위치(y 좌표)가 우선이거나, (2) 단(Column)을 먼저 구분하고 각 단 내부에서 y 좌표 순이어야 합니다.

```
def get_text_sorted(page):
    blocks = page.get_text("blocks")
    # 블록 구조: (x0, y0, x1, y1, text, block_no, block_type)

    # 정렬 전략:
    # 1. 텍스트 블록만 필터링 (block_type == 0)
    # 2. 세로(v)와 가로(h) 좌표를 기준으로 정렬
    # PyMuPDF 는 기본적으로 어느 정도 정렬을 해주지만,
    # 명확한 단 분리가 필요하다면 x 좌표를 기준으로 그룹화해야 함.

    text_blocks = [b for b in blocks if b[6] == 0]

    # 단순 정렬: 위에서 아래로, 왼쪽에서 오른쪽으로 (sort=True 옵션과 유사)
    # 복잡한 레이아웃의 경우, x 좌표의 중앙값을 기준으로 좌/우 단을 분류하는 로직 추가 필요
    text_blocks.sort(key=lambda b: (b[0], b[1]))

    return "\n".join([b[5] for b in text_blocks])
```

3.3.2 해결책 2: 분할 클리핑 (Split Clipping)

문서가 2 단임이 확실하다면, 페이지를 좌측 절반과 우측 절반으로 나누어 두 번 추출하는 것이 가장 확실한 방법입니다.²

```
half_width = page.rect.width / 2
left_rect = pymupdf.Rect(0, header_height, half_width, page.rect.height - footer_height)
right_rect = pymupdf.Rect(half_width, header_height, page.rect.width, page.rect.height - footer_height)
```

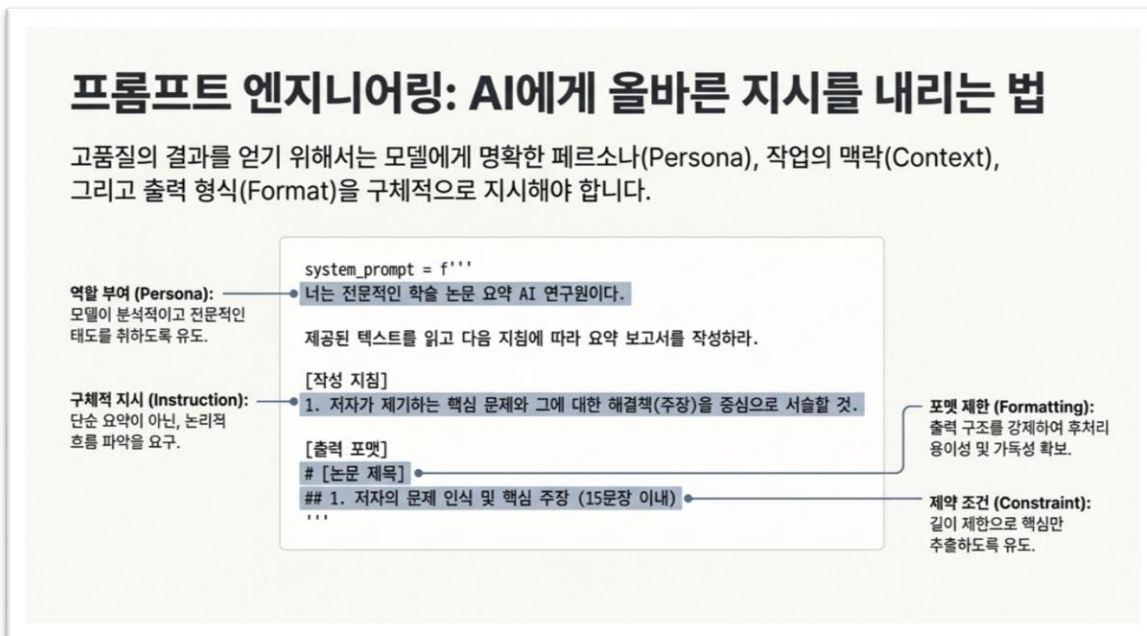
```
text_left = page.get_text(clip=left_rect)
text_right = page.get_text(clip=right_rect)
full_text = text_left + "\n" + text_right
```

이 방식은 매우 강력하지만, 1 단으로 된 초록(Abstract) 부분과 2 단으로 된 본문이 섞여 있는 페이지에서는 오작동할 수 있으므로, 페이지별 레이아웃 분석이 선행되어야 합니다.

4. 대규모 언어 모델(LLM) 기반 요약 시스템 구축

텍스트 전처리가 완료되었다면, 이제 이 텍스트를 이해하고 요약할 '두뇌'를 연결할 차례입니다. 본 강의에서는 OpenAI의 GPT-4o 모델을 사용합니다. GPT-4o는 방대한 지식 베이스와 뛰어난 맥락 이해 능력을 갖추고 있어, 복잡한 논문의 논리 구조를 파악하는 데 최적화되어 있습니다.

4.1 OpenAI API 활용의 기초와 보안



API를 사용하기 위해서는 인증 키(API Key) 관리가 필수적입니다. 소스 코드에 키를 직접 입력하는(Hard-coding) 것은 보안상 매우 위험하므로, 환경 변수 파일(.env)을 통해 관리하는 것이 표준입니다.

01_summary.py

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')

def summarize_txt(file_path: str):
```

```

client = OpenAI(api_key=api_key)

# 주어진 텍스트 파일을 읽어들이는다.
with open(file_path, 'r', encoding='utf-8') as f:
    txt = f.read()

# 요약을 위한 시스템 프롬프트를 생성한다.
system_prompt = f"""
너는 다음 글을 요약하는 봇이다. 아래 글을 읽고, 저자의 문제 인식과 주장을 파악하고, 주요
내용을 요약하라.
작성해야 하는 포맷은 다음과 같다.

# 제목
## 저자의 문제 인식 및 주장 (15 문장 이내)
## 저자 소개
===== 이하 텍스트 =====
{ txt }
"""

print(system_prompt)
print('=====')

# OpenAI API 를 사용하여 요약을 생성한다.
response = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.1,
    messages=[
        {"role": "system", "content": system_prompt},
    ]
)

return response.choices[0].message.content

if __name__ == '__main__':
    file_path = '../output/과정기반 작물모형을 이용한 웹 기반 밀 재배관리 의사결정 지원시스템

```

설계 및 구축_with_preprocessing.txt'

```
summary = summarize_txt(file_path)
print(summary)

# 요약된 내용을 파일로 저장한다.
with open('../output/crop_model_summary.txt', 'w', encoding='utf-8') as f:
    f.write(summary)
```

4.2 프롬프트 엔지니어링: 요약 품질을 결정하는 핵심 변수

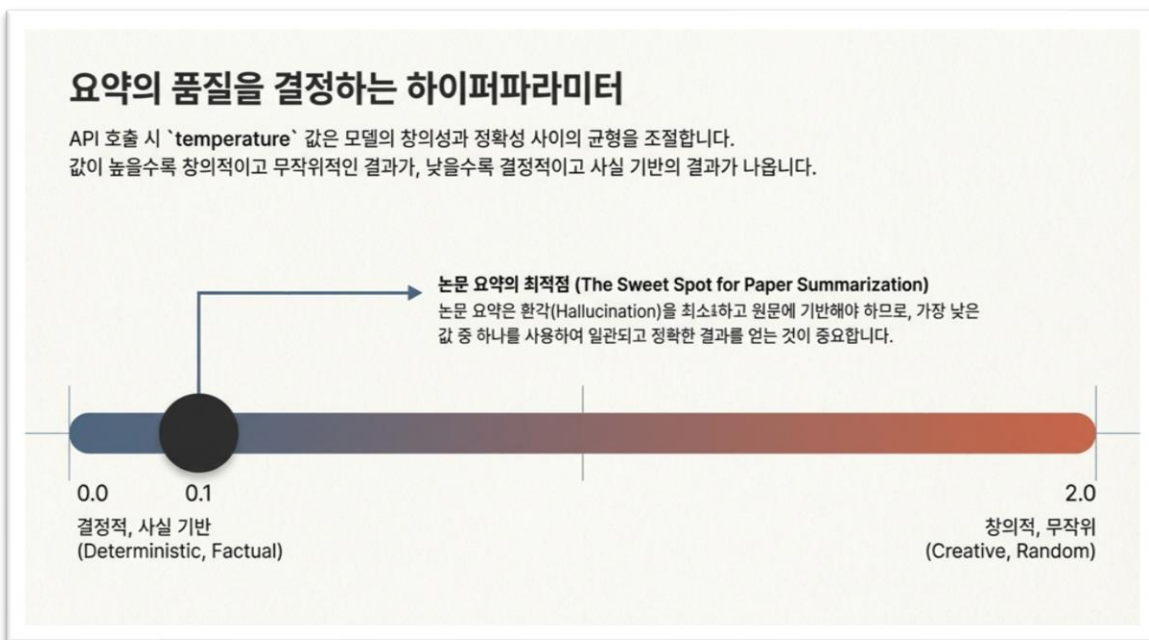
LLM에게 단순히 "요약해줘"라고 명령하는 것은 불충분합니다. 고품질의 결과를 얻기 위해서는 모델에게 명확한 페르소나(Persona), 작업의 맥락(Context), 그리고 출력 형식(Format)을 구체적으로 지시해야 합니다. 이를 **프롬프트 엔지니어링(Prompt Engineering)**이라고 합니다.⁸

4.2.1 시스템 프롬프트(System Prompt) 설계

시스템 프롬프트는 모델의 역할과 행동 지침을 정의하는 최상위 명령입니다. 강의 자료에서 제시된 프롬프트를 분석해보면 다음과 같은 우수 사례(Best Practices)가 적용되어 있습니다.

1. **역할 부여 (Persona Adoption):** "너는 다음 글을 요약하는 봇이다."라고 명시하여 모델이 분석적인 태도를 취하도록 합니다. 더 나아가 "너는 농업 공학 분야의 전문 연구원이다"라고 구체화하면 전문 용어 사용의 정확도가 높아집니다.
2. **구체적 지시 (Specific Instruction):** "저자의 문제 인식과 주장을 파악하고, 주요 내용을 요약하라." 단순히 내용을 줄이는 것이 아니라, 논문의 논리적 흐름(Problem -> Solution)을 파악하라는 고차원적인 지시입니다.
3. **포맷 제한 (Output Formatting):** Markdown 헤더(#, ##)를 사용하여 출력 구조를 강제합니다. 이는 후처리나 가독성 측면에서 매우 중요합니다.
4. **길이 제약 (Constraint):** "15 문장 이내"라는 제약을 두어 장황한 서술을 방지하고 핵심만 추출하도록 유도합니다.

4.3 하이퍼파라미터 튜닝: Temperature 와 Top_p



API 호출 시 설정하는 파라미터들은 모델의 '창의성'과 '정확성' 사이의 균형을 조절합니다.⁸

파라미터	설명	권장 설정값 (요약 업무)	이유
Temperature	다음 토큰 선택의 무작위성 조절 (0.0 ~ 2.0). 높을수록 창의적, 낮을수록 결정적.	0.0 ~ 0.2	논문 요약은 팩트에 기반해야 하며, 환각(Hallucination)을 최소화해야 하므로 가장 낮은 값을 사용하여 일관된 결과를 얻어야 함.
Top_p	누적 확률 분포 상위 p% 내에서 샘플링 (Nucleus Sampling).	0.1 ~ 0.5	Temperature 와 유사한 효과. 둘 중 하나만 조절하는 것이 일반적임.
Max Tokens	생성할 최대 토큰 수.	문서 길이에 맞게 설정	요약문이 잘리지 않도록 충분히 확보하되, 비용 통제를 위해 상한선 설정.

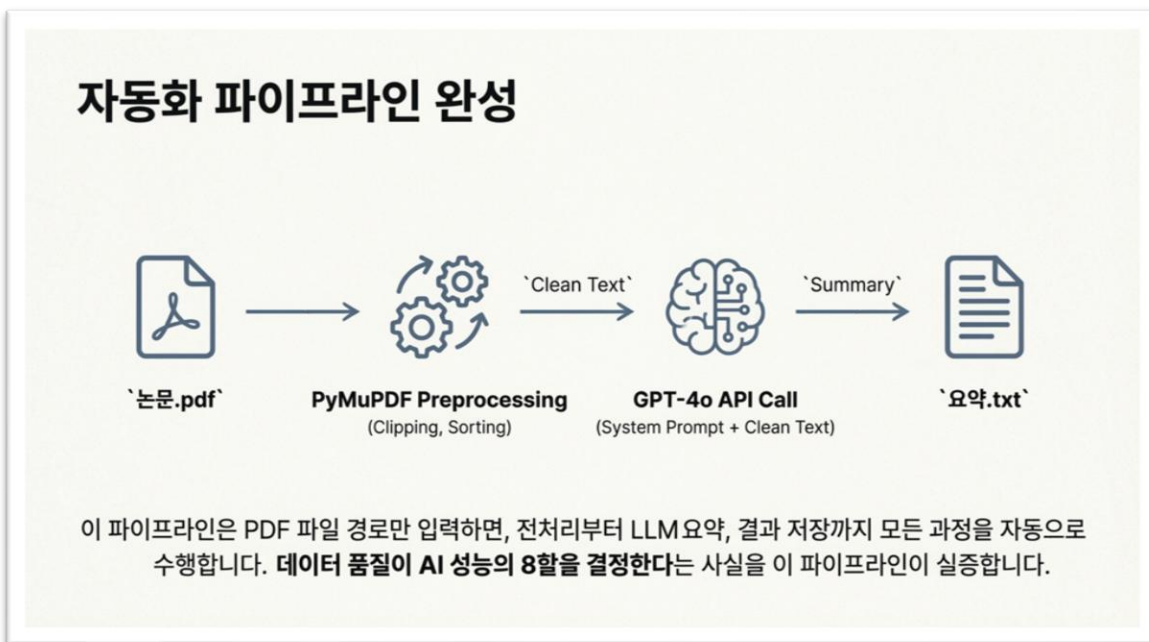
강의 자료의 코드에서 temperature=0.1 로 설정한 것은 매우 적절한 선택입니다. 이는 모델이 가장 확률이 높은 단어들 위주로 문장을 구성하게 하여, 원문의 내용을 왜곡하지 않고 건조하고 정확하게 전달하도록 강제합니다.

4.4 긴 문서 처리 전략 (Handling Long Context)

GPT-4o 는 128k 토큰이라는 거대한 컨텍스트 윈도우(Context Window)를 지원하지만, 책 한 권이나 수백 페이지의 보고서를 한 번에 처리하는 데는 여전히 비용과 성능의 제약이 따릅니다. 특히 입력이 길어질수록 중간 부분의 내용을 망각하는 '**Lost in the Middle**' 현상이 발생할 수 있습니다. 이를 해결하기 위한 전략들은 다음과 같습니다 ¹¹:

1. **Stuffing (통째로 넣기)**: 본 강의에서 사용하는 방식. 논문 1~2 편 정도의 길이에는 가장 효과적이고 문맥 유지에 유리합니다.
2. **Map-Reduce**: 문서를 챕터나 페이지 단위로 쪼개어(Chunking) 각각 요약(Map)한 뒤, 그 요약본들을 다시 하나로 합쳐 최종 요약(Reduce)을 생성합니다. 대규모 보고서 처리에 적합합니다.
3. **Refine**: 첫 번째 청크를 요약하고, 그 결과와 두 번째 청크를 함께 모델에 주면서 "이전 요약을 바탕으로 내용을 보강해줘"라고 순차적으로 요청합니다. 시간은 오래 걸리지만 문맥의 연결성이 뛰어납니다.

5. 통합 파이프라인 구축 및 실습



이제 앞서 다룬 전처리 모듈과 요약 모듈을 결합하여, PDF 파일 경로만 입력하면 최종 요약 파일이 생성되는 자동화 파이프라인(pdf_summary.py)을 완성합니다.

5.1 전체 코드 구조 및 흐름

02_pdf_summary.py

```
from openai import OpenAI
from dotenv import load_dotenv
import os
import pymupdf

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')

def pdf_to_text(pdf_file_path: str):
    doc = pymupdf.open(pdf_file_path)

    header_height = 80
    footer_height = 80
```

```

full_text = ""

for page in doc:
    rect = page.rect # 페이지 크기 가져오기

    header = page.get_text(clip=(0, 0, rect.width , header_height))
    footer = page.get_text(clip=(0, rect.height - footer_height, rect.width , rect.height))
    text = page.get_text(clip=(0, header_height, rect.width , rect.height - footer_height))

    full_text += text + '\n-----\n'

# 파일명만 추출
pdf_file_name = os.path.basename(pdf_file_path)
pdf_file_name = os.path.splitext(pdf_file_name)[0] # 확장자 제거

txt_file_path = f'../output/{pdf_file_name}_with_preprocessing.txt'

with open(txt_file_path, 'w', encoding='utf-8') as f:
    f.write(full_text)

return txt_file_path

def summarize_txt(file_path: str): # ①
    client = OpenAI(api_key=api_key)

    # 주어진 텍스트 파일을 읽어들인다.
    with open(file_path, 'r', encoding='utf-8') as f:
        txt = f.read()

    # 요약을 위한 시스템 프롬프트를 생성한다.
    system_prompt = f"""
너는 다음 글을 요약하는 봇이다. 아래 글을 읽고, 저자의 문제 인식과 주장을 파악하고, 주요
내용을 요약하라.

작성해야 하는 포맷은 다음과 같다.

```

```

# 제목

## 저자의 문제 인식 및 주장 (15 문장 이내)

## 저자 소개

===== 이하 텍스트 =====

{ txt }
'''

print(system_prompt)
print('=====')

# OpenAI API 를 사용하여 요약을 생성한다.
response = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.1,
    messages=[
        {"role": "system", "content": system_prompt},
    ]
)

return response.choices[0].message.content

def summarize_pdf(pdf_file_path: str, output_file_path: str):
    txt_file_path = pdf_to_text(pdf_file_path)
    summary = summarize_txt(txt_file_path)

    with open(output_file_path, 'w', encoding='utf-8') as f:
        f.write(summary)

if __name__ == '__main__':
    pdf_file_path = "../data/과정기반 작물모형을 이용한 웹 기반 밀 재배관리 의사결정 지원시스템

```

설계 및 구축.pdf"

```
summarize_pdf(pdf_file_path, '../output/crop_model_summary2.txt')
```

5.2 실행 결과 분석과 인사이트

실제 예제 논문을 대상으로 파이프라인을 실행하면, 전처리가 적용되지 않았을 때와 비교하여 확연히 다른 결과를 얻을 수 있습니다.

- **전처리 전 (Raw Text):** 요약문에 "한국농공학회 66 권에 따르면..."과 같이 불필요한 출처 정보가 반복되거나, 페이지 번호가 데이터 수치인 것처럼 오인되어 포함될 수 있습니다.
- **전처리 후 (Clean Text):** "저자는 밀 자급률 1% 미만의 현실을 지적하며..."와 같이 논문의 논리적 흐름에 집중된 깔끔한 요약이 생성됩니다. 이는 **데이터 품질이 AI 성능의 8 할을 결정한다**는 사실을 실증합니다.

6. 심화 주제: 시스템 확장 및 미래 전망

본 강의에서 구축한 시스템은 가장 기초적인 형태의 'AI 연구원'입니다. 이를 실제 현업 수준의 프로덕션(Production) 시스템으로 확장하기 위해 고려해야 할 고급 주제들을 논의합니다.

6.1 스캔된 PDF 처리: OCR 통합

많은 오래된 논문이나 행정 문서는 텍스트가 아닌 이미지 형태로 스캔되어 저장된 경우가 많습니다. PyMuPDF 만으로는 이러한 문서에서 텍스트를 추출할 수 없습니다. 이 경우 **OCR(광학 문자 인식)** 기술을 파이프라인 앞단에 추가해야 합니다.

- **Tesseract**: 오픈소스 OCR 엔진으로, PyMuPDF 와 연동하여 이미지 영역의 텍스트를 추출할 수 있습니다.²
- **GPT-4o Vision**: 최신 모델은 이미지를 직접 보고 내용을 이해할 수 있으므로, 텍스트 추출 과정 없이 PDF 페이지 이미지를 바로 모델에 입력하여 요약하는 **멀티모달(Multi-modal)** 접근 방식도 가능해졌습니다. 이는 표나 그래프의 데이터까지 해석할 수 있다는 강력한 장점이 있습니다.

6.2 검색 증강 생성 (RAG) 시스템으로의 발전

단일 문서 요약을 넘어, 수백 편의 논문 데이터베이스에서 질문에 대한 답을 찾는 시스템을 구축하려면 **RAG (Retrieval-Augmented Generation)** 아키텍처가 필요합니다.

1. **임베딩(Embedding)**: 추출한 텍스트를 벡터(숫자 배열)로 변환하여 벡터 DB(예: Pinecone, Chroma)에 저장합니다.
 2. **검색(Retrieval)**: 사용자의 질문과 유사도가 높은 문서 조각(Chunk)을 검색합니다.
 3. **생성(Generation)**: 검색된 조각들을 LLM 에 참고 자료로 제공하여 답변을 생성합니다.
- 본 강의의 텍스트 추출 및 전처리 모듈은 RAG 시스템의 데이터 수집(Ingestion) 단계로서 완벽하게 재사용될 수 있습니다.

6.3 멀티턴(Multi-turn) 대화형 인터페이스

현재 시스템은 문서를 한 번 읽고 요약하는 '싱글턴(Single-turn)' 방식입니다. 이를 챗봇 형태로 발전시켜 "이 논문의 실험 방법론 부분만 더 자세히 설명해줘"와 같이 꼬리에 꼬리를 무는 질문을 처리하려면, 대화의 히스토리(History)를 관리하는 '멀티턴' 구조로 변경해야 합니다.¹² 이를 위해서는 messages 리스트에 사용자의 질문과 AI 의 답변을 계속 누적하여 API 에 전달하는 로직이 추가되어야 합니다.